



Nordheim Online

PROOF OF CONCEPT · V1

NORDHEIM ONLINE × DIGITALOCEAN

Drakkar Lite on DigitalOcean Kubernetes

Deployment, results and recommendations



Chuck Talk

Technical Account Manager · September 2026

Nordheim Online set four goals for the move to Kubernetes



Scalability

Handle growing workloads as new customers onboard



Performance

Deliver fast response times to every tenant



Reliability

Stay available with minimal downtime, including during upgrades



Cost

Avoid unnecessary cloud spend on a single-developer budget

WHAT WAS DELIVERED

A live, autoscaling SaaS app over HTTPS

DOKS cluster

Kubernetes 1.36 in NYC3, HA control plane,
2 nodes autoscaling to 3

Application

2 web pods and 2–5 API pods in Docker
containers, CPU-based autoscaling

Load balancer

One DigitalOcean Load Balancer, created by
a Gateway that routes web and API

Database

Managed PostgreSQL on the private
network, encrypted, backed up daily

HTTPS

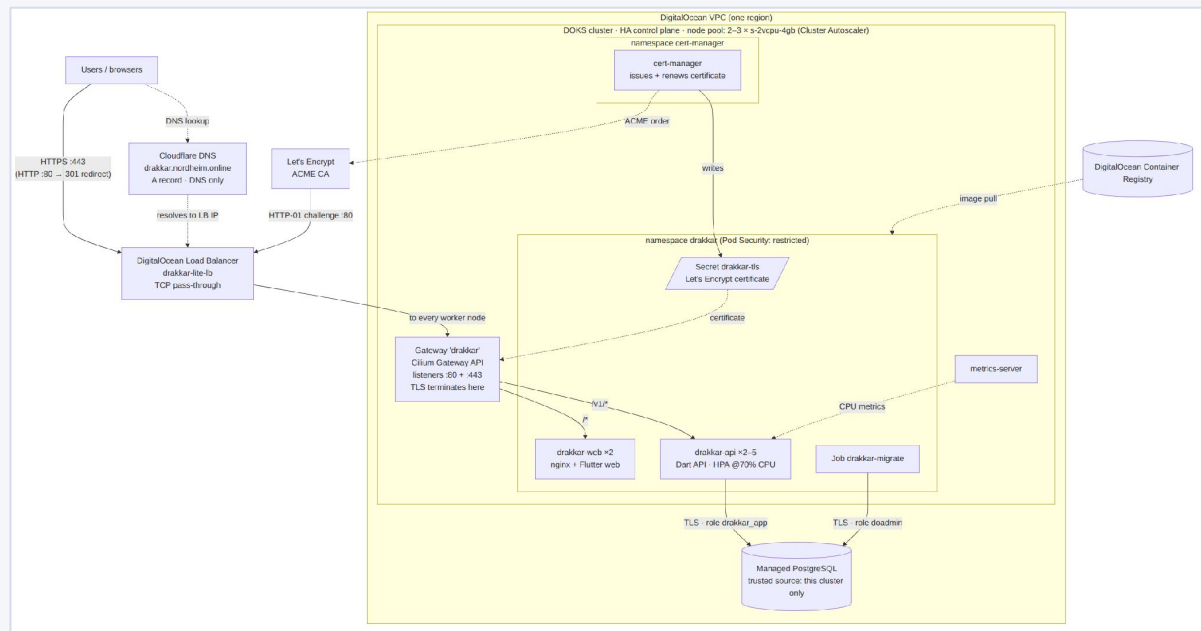
Free Let's Encrypt certificate, renewed
automatically by cert-manager

Tenant isolation

Each organization sees only its own data,
enforced by the database

ARCHITECTURE

One front door, one private database



Request path

Browser → Load Balancer → Gateway
(TLS) → web or API pod

Data path

API → private network → PostgreSQL,
TLS required, this cluster only

Certificate

cert-manager ↔ Let's Encrypt;
Cloudflare is DNS only

Every release follows the same five steps

1

Build

Test and package both images for linux/amd64

2

Push

Upload to the private DigitalOcean registry

3

Migrate

A one-time job updates the database first

4

Deploy

Pods replaced one at a time, no downtime

5

Verify

Isolation, rollout, load and cost checks

Scripted today, with no CI/CD bill. Automating it is Nordheim's choice (recommendation R9).

The API tripled its capacity in 15 seconds

2 → 5

API pods under load (the set maximum)

60 s

to scale back to 2 once load ended

Time	API CPU vs target	Pods	What happened
16:02:11	47% / 70%	2	Load starts
16:02:26	842% / 70%	2	Target crossed
16:02:41	858% / 70%	5	Scaled out
16:06:42	1% / 70%	5	Load ends
16:07:42	1% / 70%	2	Scaled back

Well inside the 300 ms target, with zero errors

P95

132

ms

144 ms on the second run

P99

229

ms

Median 84–87 ms

THROUGHPUT

39.9

requests per second,
sustained

ERRORS

0.00%

0 of 24,048 requests failed

Two 5-minute k6 runs over HTTPS, ramping to 50 simulated users, each signing up and reading contacts.

Four failure tests, zero failed requests

Test	What we did	Result
Rolling update	Replaced every API pod under live traffic	275 requests, 0 failed
Self-healing	Deleted a running pod	205 requests, 0 failed
Node drain	Emptied a server, as in an upgrade	269 requests, 0 failed
Rollback	Reverted to the previous release	Completed cleanly

Also in place: HA control plane with a 99.95% SLA, disruption budgets that keep 1 API and 1 web pod up, health probes, daily database backups.

Customer data is protected at every layer



HTTPS everywhere

All 13 TLS checks passed; TLS 1.1 refused; HTTP redirects to HTTPS



Tenant isolation

Row-level security enforced by PostgreSQL; cross-tenant read returns 404



Private database

Reachable only from this cluster, over the private network, TLS required



Hardened containers

Non-root, read-only file systems, restricted Pod Security; passwords bcrypt-hashed

GOAL 4 · COST

\$123.15 a month, capped at \$147.15

Resource	Monthly
HA control plane	\$40.00
2 × s-2vcpu-4gb nodes	\$48.00
Load balancer	\$15.00
Managed PostgreSQL, 1 GiB	\$15.15
Container registry, Basic	\$5.00
HTTPS certificate and DNS	\$0.00
Baseline	\$123.15
Third node, only when busy	+\$24.00

How the design avoids waste

- One load balancer serves web and API
- Autoscaling capped at 5 pods and 3 nodes
- Small, measured resource requests
- Idle nodes run at 5–6% CPU, about a third of memory
- HTTPS added \$0
- Hourly billing and a scripted teardown

Nordheim can rebuild everything without help

Part	What it covers
1	Prepare the computer and tools (steps 1–8)
2	Create the DigitalOcean resources (9–13)
3	Put the app on the cluster (14–18)
4	Turn on HTTPS (19–21)
5–6	Final checks and everyday care
7–8	Troubleshooting and shutting down

Every step, the same shape

What you're doing → Why → Do this → Check it worked

About 2 hours

Written for someone who can copy and paste into a terminal

All four goals met in the proof of concept

Goal	Status	Evidence
Scalability	Met	2 → 5 API pods in 15 s; back to 2 in about 60 s
Performance	Met	p95 132–144 ms, p99 229 ms, 0% errors
Reliability	Met	0 failed requests in rollout, self-heal and drain
Cost	Met	\$123.15 a month; HTTPS added \$0

Open items: pod spreading (R3); optional tests at full scale and with a third node (R4).

Your decisions, and what each would cost

#	Recommendation	Monthly	Risk it addresses
R1	HTTPS with automatic renewal: closed	\$0	Plain HTTP
R2	Keep the HA control plane (already on)	\$40, in run rate	Cluster outages
R3	Require API and web pods to spread across nodes	\$0	Single-node failure
R5	Database standby (2 GiB primary + standby)	About +\$45	Database failure
R6	PgBouncer pooling before raising the pod limit	\$0	Connection limit
R7–R8	NetworkPolicies; Sealed or External Secrets	\$0 or provider	Lateral movement; secrets
R9	CI/CD: keep scripts, GitHub Actions or own runner	\$0 options	Release mistakes
R10	Alerts, and auto-upgrades in a maintenance window	\$0 for alerts	Version drift

NEXT STEPS

Three decisions for Nordheim Online

1

Approve the \$0 fixes: R3, R6, R7, R10

2

Choose secrets (R8) and CI/CD (R9) approaches

3

Decide when to add the database standby (R5)

Questions?

Chuck Talk · chuck@nordheim.online
github.com/TaliskerMan/drakkar-lite-doks